

Fault Tolerant University Information System

Midterm technical report | Fault Tolerance and Dependable Computing

Campus reliability lab | 5 October 2026

Student name: Tastan Magzhan

Group: CSE-2505M

Instructor: Azamat Serek

A compact, executable university platform compares an intentionally weak baseline with a fault-tolerant configuration. Three business services and one shared SQLite owner expose failures, recovery and transaction consistency through real HTTP experiments.

The submitted dataset contains 54 controlled runs: nine scenarios, two configurations and three trials. Measurements use local processes on one Windows computer. Physical node failure is modelled by stopping a group of processes; it is not a multi-host hardware test.

Repository: https://github.com/zzmaga/ftu_midterm

Evidence: results/final | Reproduction: README.md | Demonstration: docs/DEMO.md

1 Introduction and project motivation

A university platform should not duplicate tuition payments when a reply is lost, nor make payment access depend on an optional transcript service. This project demonstrates both availability and integrity. A small architecture makes each protection visible in code and supports a short oral demonstration.

The main result is fault containment: a second payment process preserves payment access after one replica fails, while transactions and idempotency prevent partial or repeated payment effects. A shared database outage still interrupts all database-dependent operations. Redundancy is therefore useful only within its actual failure boundary.

2 System requirements and assumptions

The assignment requires a baseline, at least three independent services, at least five realistic faults, two infrastructure mechanisms, four software mechanisms, controlled before/after experiments, reliability calculations and a demonstration of three recovery scenarios. All six explicitly listed scenarios are implemented, with three additional integrity/recovery exercises.

Teaching acceptance criteria are: no failed payment-list requests during a single payment-replica crash in FT; zero balance/ledger discrepancies and duplicate keys in all FT cases; restored full responses after repair; and recorded detection, restoration and request evidence. These are experiment-specific criteria, not a production SLA.

Two fictitious students start with 100000 tiyn tuition due each. Amounts use integer tiyn (100 tiyn per KZT). Every experiment first commits an unmeasured 100-tiyn seed payment, so persistence is checked on non-empty data. Fault controls are local and token protected. Authentication, real bank integration and multi-host deployment are outside the scope.

3 Dependability and fault model

Dependability here includes availability, integrity and recoverability. Reliability concerns uninterrupted service over an interval; availability permits repair and counts the usable fraction. A fault is a cause, an error is an incorrect internal outcome, and a failure is a violation of the service contract.

Example: termination of payment1 is a fault; connection refusal at the gateway is an error; an unsuccessful client payment request is a service failure. In FT the gateway can route to payment2, so the error is masked. Lost acknowledgement is an omission fault: a payment may already exist even though the client observed failure.

Faults are crash-stop, transient delay/error, omission after commit and an injected workflow interruption. Grouped process loss models a permanent node fault until explicit repair. Byzantine peers, arbitrary disk corruption, simultaneous host loss and malicious requests are excluded. A degraded transcript is useful but does not satisfy the full-response availability definition.

4 Reliability and failure analysis

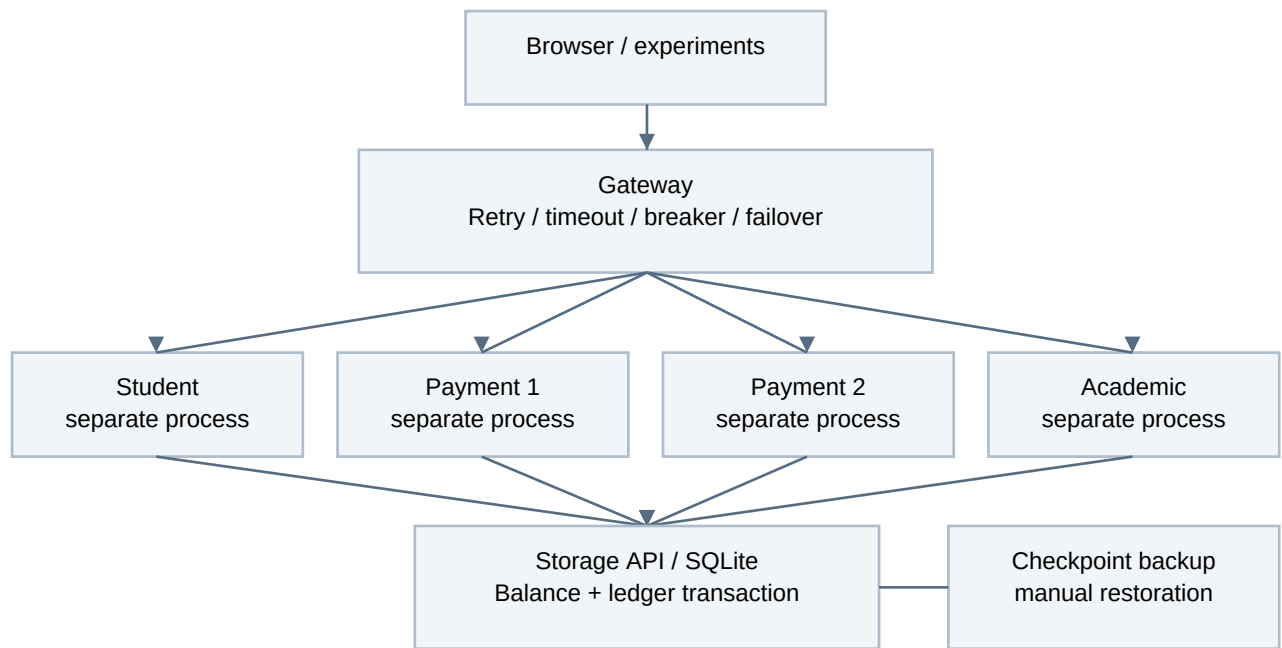
The qualitative failure analysis below traces each injected fault to its client impact and remaining risk. Severity is descriptive; no unsupported occurrence probabilities or risk-priority scores are assigned.

Fault	Error and service effect	Protection and residual risk
Application crash	Payment endpoint cannot connect to payment1.	Replica and failover preserve access; gateway and database remain single points.
Database outage	Storage connection fails; business APIs cannot read/write.	Bounded retry and controlled 503; explicit restart restores access. No database replica.
Service timeout	Academic reply exceeds the downstream timeout.	Breaker limits calls; labelled cached response. Full transcript is unavailable.
Node A simulation	Payment1 and academic are killed together.	Payment2 survives; student works. Whole-host failure remains unprotected.
Interrupted payment	Balance changes before ledger insertion.	FT transaction rolls back; baseline exposes a discrepancy.
Concurrent load	Queueing and shared storage contention increase latency.	Two workers distribute traffic; shared SQLite serializes writes.
Duplicate or lost reply	A payment request is delivered more than once.	Shared unique key and payload check return the original payment.
Checkpoint restore	Operator restores an earlier database state.	Consistent backup recovers that state but loses post-checkpoint writes.

A simple fault tree for payment unavailability is: gateway failure OR storage failure OR (payment1 unavailable AND payment2 unavailable) OR a common host/network failure. For baseline, one payment failure alone is sufficient. Integrity failure has separate causes: missing atomicity, missing idempotency or recovery to an older checkpoint without reconciliation.

Under an illustrative independence assumption, two replicas each with availability $A=0.99$ have combined availability $1-(1-A)^2=0.9999$. If the gateway and storage each also have 0.99 availability, the series product is $0.99 \times 0.99 \times 0.9999 = 0.98000199$. These are hypothetical values, not measured estimates; a shared host also violates independence.

5 System architecture



All services emit JSON events. The harness records requests, faults and audits.

Figure 1. Each business service runs as an independent HTTP process. FT adds payment2. The gateway is a small FastAPI application rather than an external reverse proxy, making routing decisions and circuit state easy to inspect. Mermaid source is supplied in docs/architecture.md.

Ports are 8100 for the gateway, 8101 for students, 8102 and 8105 for payments, 8103 for academic records, and 8104 for storage. Experiments use 8200-8205; tests use 8400-8405. The launcher checks that ports are free and owns only its child process trees.

All business services access the same storage API. Only its process opens SQLite. This single transaction authority makes payment keys consistent across replicas. The gateway holds a per-endpoint breaker and an in-memory transcript cache; cache loss after gateway restart is acceptable and explicitly produces a degraded response without cached data.

The simulated node A contains payment1 and academic; node B contains payment2 and student. Both are logical groups on one computer. JSON logging is integrated into each process, while the experiment harness collects request timing and state snapshots separately.

6 Hardware fault tolerance design

Infrastructure protection consists of redundant payment instances, round-robin routing with failover, and backup restoration. A killed payment process is bypassed by the gateway. The local experiment harness later restarts it; this restart is scheduled by the experiment, not an autonomous production orchestrator.

SQLite backup creates a consistent checkpoint file [2]. The restore experiment deliberately removes a valid later payment by returning to the checkpoint, making recovery-point loss explicit. This is logical storage recovery on one disk, not independent storage redundancy. A production design would copy backups to another fault domain and reconcile later transactions.

RAID can use disk redundancy to tolerate selected disk faults; it does not undo a logically incorrect write or replace backups. ECC memory can detect and correct supported memory bit errors; it does not repair an application algorithm. Both concepts are documented only. Neither RAID nor ECC is claimed as implemented or measured.

7 Software fault tolerance design

Mechanism	Implementation and boundary
Retry and timeout	Gateway permits at most 3 attempts, with 50/100 ms backoff and a 300 ms HTTPX timeout. Baseline uses 1 attempt and a 1 s safety timeout [3].
Circuit breaker	Two downstream failures open a per-endpoint circuit. After 1 s, one HALF_OPEN probe can close it or reopen it [5].
Idempotency	A unique SQLite key is checked inside BEGIN IMMEDIATE. Same key and payload return the existing row; conflicting payload receives 409.
Checkpoint and rollback	A savepoint precedes balance change; interruption rolls back. The outer transaction commits balance and ledger together [1].
Health and degradation	Liveness routes expose process status. Academic failure returns explicitly marked cached data; payments never fall back to invented success.

8 Implementation

Python, FastAPI, HTTPX and SQLite keep the application small. `app/service.py` is launched independently for three service roles. `app/database.py` owns storage and auditing; `app/resilience.py` owns recovery policy. The baseline deliberately omits idempotency and commits the balance before inserting its payment row. Both variants retain validation.

Meaningful automated tests cover concurrent cross-replica duplicates, conflicting payloads, lost replies after commit, exact rollback state, backup restore, breaker transitions, non-retried 4xx responses and reliability calculations. Seventeen tests passed. Compose configurations validate successfully; container runtime execution was unavailable because the Docker engine was not running [4].

9 Experimental methodology

Execution environment: Windows-11-10.0.26300-SP0; Python 3.14.6. Dataset start (UTC): 2026-10-05T09:41:17.045013+00:00. The workload ran against real local HTTP processes. Three trials per scenario and mode produce 54 isolated runs; baseline/FT execution order alternates by trial.

Each run has a new database directory. A seed payment is committed, four warm-up probes are recorded, and then the fault is injected. Crash/database/node cases make seven affected-period probes, restart owned processes, wait for liveness and a 1.05 s breaker cooldown, then make five recovery probes. Probes sleep 80 ms between completed requests; timeout cases have five affected and five recovery probes and a 1.1 s post-repair wait.

The timeout fault adds 1400 ms of server delay. The node fault kills payment1 and academic together. The load case issues 120 unique 100-tiny payments with concurrency 30; duplicate uses twelve simultaneous deliveries of one key. Interruption fires once between balance change and ledger insertion. Lost response returns a simulated 503 after commit. Backup restores an earlier consistent checkpoint and verifies the exact audit state.

Measurement	Operational definition
Detection time	Injection event to first observed downstream failure, storage error or rollback log. Wall clocks share one host. No observed signal is n/a.
Service recovery	Injection to first subsequent full successful probe. If failover masks the fault, this is time to first confirming success, not outage duration.
Component restoration	Injection to the harness's completed restart/reset marker. This is distinct from successful service recovery.
Request availability	Full successful responses / measured requests. Degraded academic HTTP 200 responses are excluded and counted separately.
Consistency	initial_due - due equals SUM(payment.amount), no duplicate keys, and the seed payment remains. Baseline HTTP success alone can hide corruption.
Recovered requests	Full successful client requests that required more than one gateway attempt. No claim is made about unobserved downstream retries.

Timing uses perf_counter for request/event intervals; structured logs use a UTC wall clock to correlate failure detection. Request time availability is sampled, not continuously observed. Concurrent load requests do not define outage intervals; pre/post probes can miss an outage within the burst. The report therefore interprets time metrics only for crash, database, timeout and node cases.

10 Results and comparison

Scenario	Baseline full	FT full	B consistent	FT consistent
Application crash	27/48	48/48	3/3	3/3
Database outage	27/48	27/48	3/3	3/3
Academic timeout	33/48	33/48	3/3	3/3
Node A loss	30/54	51/54	3/3	3/3
Interrupted payment	27/30	30/30	0/3	3/3
Concurrent load	384/384	384/384	3/3	3/3
Duplicate delivery	60/60	60/60	0/3	3/3
Checkpoint restore	30/30	30/30	3/3	3/3
Lost acknowledgement	27/30	30/30	0/3	3/3

Table 1. Counts combine three trials. Consistent columns count trials meeting both business invariants. Extra student/payment checks in the node and timeout scenarios are included in the response totals. Backup is a deliberate recovery exercise; baseline has no checkpoint restoration facility.

Across the mixed workload, baseline had 87 failed full responses out of 732; FT had 39 out of 732. FT recovered 18 requests within retries. These totals summarize this scenario mix and should not be interpreted as a general availability SLO.

Replica failover eliminated payment-list failures during application crash. FT node loss still degraded the academic endpoint, which is deliberately counted as loss of full service. Both modes lost full database-dependent responses during storage outage; neither has storage replication. Baseline duplicate and lost-reply cases returned successful HTTP responses while violating transaction identity, showing why request availability and integrity must be evaluated separately.

Mode	Mean requests/s	Trial range	Mean case p95 ms
baseline	114.0	65.8 - 150.6	384.4
ft	164.4	105.4 - 205.5	225.5

Table 2. Throughput counts the 120 completed burst requests; all burst requests succeeded in the submitted dataset. The p95 column is the mean of each case's nearest-rank latency percentile, including its pre/post probes. Three trials do not establish a statistically robust performance advantage.

Reliability calculations for Section 10

Let T be the observation span from first request start to final audit; D be the union of probe-observed outage intervals; $U=T-D$; F be the number of outage episodes; and C be completed recovery episodes. An outage begins at completion of the first failed full probe and ends at completion of the first subsequent full success. An open final outage is censored at T.

MTTF estimate = U/F ; MTTR = $\text{sum}(\text{completed outage durations})/C$; MTBF estimate = MTTF + MTTR; time availability = U/T ; observed failure rate = F/U . Request availability = full successes/N; retry recovery fraction = successful retried requests/all retried requests. Missing denominators produce n/a. The repairable-cycle MTTF/MTBF values include finite-window exposure, so they are descriptive induced-failure estimates, not natural lifetime measurements.

Scenario	Mode	MTTF s	MTTR s	MTBF s	Time A	Rate /s
crash	baseline	2.004	8.708	10.713	18.7%	0.499
crash	ft	n/a	n/a	n/a	100.0%	0.000
database	baseline	1.811	7.537	9.348	19.4%	0.552
database	ft	2.086	3.014	5.100	40.9%	0.479
timeout	baseline	1.897	5.650	7.547	25.1%	0.527
timeout	ft	1.662	1.607	3.268	50.8%	0.602
node	baseline	2.182	9.945	12.127	18.0%	0.458
node	ft	n/a	n/a	n/a	100.0%	0.000

Table 3. Pooled exposure and recovery durations over three trials. FT node time availability uses payment probes; its separate degraded transcript request is counted in Table 1, not as a payment outage. FT crash and node therefore have no observed payment outage from which to estimate MTTF or MTTR.

Scenario	B detect ms	FT detect ms	B confirm ms	FT confirm ms
crash	1139.5	544.0	9849.4	145.6
database	934.7	437.5	8474.9	4234.2
timeout	1013.7	317.1	6665.6	2402.2
node	1293.4	685.1	11240.2	293.3
interruption	14.1	10.6	41.6	82.8
response_loss	19.1	13.7	41.0	83.0

Table 4. Mean injection-to-detection and injection-to-first-success intervals. They include workload scheduling. Longer baseline timeout probes can also delay the script's scheduled repair, so recovery differences are partly a harness/workload effect and must not be attributed entirely to faster infrastructure repair.

11 Discussion and limitations

The experiment-specific availability and integrity goals were met for the selected FT cases: every single-replica crash retained full payment responses, all FT audits were consistent, and normal probes resumed after repairs. The dataset does not justify a broader claim of continuous service under database or whole-host loss. A real deployment would need a redundant gateway, database replication, separate hosts and a tested backup recovery policy.

The load test is bounded and the database serializes payments under a process lock. Replication therefore mainly addresses process failure, not write scaling. The baseline intentionally has weak transaction boundaries; its difference from FT is a pedagogical contrast rather than a comparison with a competently designed production payment system.

Timeout and retry settings were selected for a local demonstration. HTTPX timeouts constrain network phases, not a strict end-to-end deadline [3]. The service-to-database timeout remains 800 ms. Retries occur only at the gateway to avoid multiplicative retry storms. Production work should add jitter, load shedding, an overall deadline, bounded caches and request tracing across hosts.

Circuit state and transcript cache live in one gateway process. Health endpoints prove liveness, not database readiness. Failure detection is passive when traffic encounters a broken endpoint. The node simulation does not reproduce CPU, kernel, disk or network-switch failures. Physical hardware redundancy and RAID/ECC were not measured.

Only three repetitions, one host, finite workloads and intentionally injected failures are available. No confidence interval or natural failure-rate claim is supported. Probe estimates miss the interval before the first failure is observed and may miss short outages between probes. No unmeasured outage during a load burst is silently treated as proven availability.

Restore discards post-checkpoint payments and keys. It must be performed during a maintenance/quiescence window and followed by reconciliation. The demonstration's backup sits on the same disk. The application is a local teaching tool with unauthenticated business endpoints and must not be publicly exposed.

12 Conclusion

A small university system is sufficient to demonstrate observable fault tolerance. Application replication protects a selected service from process loss; a breaker contains repeated dependency failures; transaction rollback and idempotency preserve payment integrity. The experiments also show the limits: a shared database and host still define a common failure boundary. Raw requests, event logs and audit snapshots make these claims reproducible.

13 References

[1] SQLite. Atomic Commit in SQLite. Accessed 5 October 2026.

<https://www.sqlite.org/atomiccommit.html>

[2] SQLite. Online Backup API. Accessed 5 October 2026.

<https://www.sqlite.org/backup.html>

[3] HTTPX. Timeouts. Accessed 5 October 2026.

<https://www.python-httpx.org/advanced/timeouts/>

[4] Docker. Control startup and shutdown order in Compose. Accessed 5 October 2026.

<https://docs.docker.com/compose/how-tos/startup-order/>

[5] Microsoft Learn. Circuit Breaker pattern. Accessed 5 October 2026.

<https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker>

[6] FastAPI. Deployment concepts. Accessed 5 October 2026.

<https://fastapi.tiangolo.com/deployment/concepts/>

The primary documentation supports the mechanisms and deployment choices [1-6]. All measured values come from this repository's experimental records, not from the reference examples.

14 Appendix source code configuration logs and additional results

Artifact	Contents
app/	Gateway, shared fault controls, business services, resilience policy, database and dashboard.
scripts/	Owned-process launcher; reproducible experiments; metrics; evidence verification; report builder.
tests/	17 automated unit and HTTP integration tests.
compose.yaml and baseline override	Container topology, liveness checks, replicas, separate bind-mounted datasets.
results/final/	54 JSON cases, summary.csv, environment metadata and per-service JSON event logs.
docs/	Architecture diagram, assignment checklist, demo and defense notes, generated report.

Reproduce: `python -m pytest -q`; `python -m scripts.experiments --trials 3`; `python -m scripts.verify_results results/final`. Experiments write new folders, leaving the submitted dataset unchanged. Rebuild this report with `python -m scripts.build_report --dataset results/final` after installing `requirements-report.txt`.

Source logs record request IDs, UTC timestamps, service instance, mode, action, status and latency. Transaction events carry their payment key. Fault, attempt and recovery fields appear on applicable events. Missing fields do not imply fabricated measurements.